Implementation and Analysis of BST, AVL and RBT Data Structures

Sebastiano Babich (172249) - Leonardo Mazzon (172587)
Damiano Netto (171948)

University of Udine, Academic Year 2025 - 2026

Contents

1 Abstract

The goal of the project is to implement and compare some of the data structures studied in the "Algorithms and Data Structures" course at the University of Udine, specifically standard and self-balancing binary search trees. The project focuses on analyzing the time complexity of insertion and deletion within these structures. We also analyze the heights and rotation performed by the trees of our implementation.

2 Introduction

Binary search trees (BST) are among the most widely used data structures, since they yield logarithmic asymptotic time in the average case of search, insertion and deletion. As the name suggests, each node in the tree has to have two children, which can either be another node or a null pointer, and the tree's height is the number of edges in the longest path between the root of the tree and a leaf. Two examples of Red-Black Trees (balanced variation of BSTs, covered in our analysis) being used are `std::set`[2] in C++'s standard library and the `Completely Fair Scheduler`[1] in Linux.

3 Theoretical Background

In all tree data structures, height is a fundamental property: keeping its value as small as possible helps ensure efficient operations thanks to more efficient search, insertion and deletion operations.

Trees can be divided into two categories based on how they manage their height: **binary search trees** and **balanced binary trees**. Standard binary search trees do not actively control their height, while balanced binary trees use specific rules and rotation operations to keep the height as small as possible.

Trees of the latter kind are designed to maintain a height close to the order of $\log_2(n)$, where n is the number of nodes inserted into the tree.

In this project, we studied three types of trees, two of which implement self-balancing techniques.

3.1 Binary Search Tree (BST)

Binary search trees do not impose any restrictions on their height. Their purpose is to provide efficient search, insertion and deletion operations maintaining an ordered structure of the stored elements. However, since they do not include any mechanism to balance them, the tree can become unbalanced depending on the order in which nodes are inserted. If for example, nodes are inserted in an orderly manner, it will result in the Worst-Case complexity (which is listed below in asymptotic time), making the tree isomorphic to a linked list. So inserting values in a "random" order will yield better results.

Operation	**Average Case**	**Worst Case**
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$

Table 1: Time complexity in BST operations

3.2 Adelson-Velsky Landis Tree (AVL)

AVL trees are a data structure that implements self-balancing binary search trees. The search operation in the tree exactly mirrors the search in BST. Insertion initially uses the BST insertion method, with the difference that a self-balancing rotation is performed afterwards.

Specifically, this operation aims to identify imbalances in the tree by comparing the heights of two child nodes. Specifically, the goal is to keep total height as low as possible, having an upper bound of $1.44 \cdot \log_2(n+2)$(**adelson-velsky1962**). This yields better time complexity in search operations, however, its main drawback is insertion time, as rotations provide a lot of overhead, but this is necessary to achieve the aforementioned height.

Operation	**Average Case**	**Worst Case**
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 2: Time complexity in AVL operations

3.3 Red Black Tree (RBT)

An RBT is a self-balacing BST where every node is colored as red or black and satisfies certain conditions. So for each node we also have to keep track to its color. If T is a RBT, for all n node of T, we call $color(n)$ its color.

Let T a BST where each node has a color value associated ($color(n)$).

T is a RBT $\iff$ T satisfies the following conditions:

- For all n node in T, $color(n)$ must be either BLACK or RED (it must be a total function from the set of all nodes of T to {BLACK, RED})
- NIL leaves are considered black.
- For all red nodes n in T, n must not have a red child.
- Every path from a n node in T to its descendant NIL leaves must contain the same number of black nodes.

For readonly operations (like search, inorder visit, etc...) we have the same complexity as BST. Instead, insertion and removal operations are augmented using two new helper functions whose main purpose is to mantain the properties of the RBT, using rotations like AVL Trees.

These trees are a compromise between the aforementioned trees, as they yield efficient results even in insertion and deletion, due to their limited use of rotations during the balancing of the tree. Although this means that the tree might not achieve is absolute minimum height, as it has an upper bound of $2 \cdot \log_2(n+1)$ (**cormen2022**), which is slightly worse than the height of the AVL. Since this tree combines fast insertion akin to a BST and a search almost as optimal as an AVL. It is suitable in systems requiring frequent search and insert/delete operation

Operation	**Average Case**	**Worst Case**
Search	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
Delete	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Table 3: Time complexity in RBT operations

4 Implementation

The data structures (BST, AVL and RBT) were implemented as separate modules in the project, with similar interfaces for insertion, deletion, and search. Each tree maintains additional information useful for analysis, such as current height and number of rotations. Balancing operations were separated into dedicated functions within each type of tree to comply with the rules imposed by the data structure used.

These are the main functionalities implemented for each tree:

BST (Binary Search Trees)

- **Node implementation**: a *Node* class has been implemented to store the key, the references to the left and right children, and a reference to the parent node. Each one of these could be another node, or null reference.
- **Node insertion and deletion**: insertion and deletion operations are implemented according the standard algorithms of Binary Search Trees, maintaining the BST ordering property.
- **Search operations**: efficient search, minimum, maximum, predecessor and successor operations are provided by exploiting the ordering property of the Binary Search Tree.
- **Tree traversals operations**: inorder, preorder and postorder are implemented iteratively to permit the visit of each tree created.
- **Tree rotations**: left and right rotation operations are implemented as fundamental tree transformations. These operations are reused by the balanced tree implementations and a rotation counter is kept to be used in the experimental analysis.

AVL (Adelson-Velsky Landis Trees)

- **AVLNode implementation**: an *AVLNode* class, extending the *Node* class contained in BST, has been created. Each node stores an additional *height* field, which is required to compute the balance factor.
- **Balance factor computation**: the balance factor of each node is calculated as the difference between the heights of its left and right subtrees. That value is used to detect unbalanced configurations.

- **Height management**: the height of each node is updated after insertions, deletions and rotations through the *update_height* function, avoiding the need of recomputing heights recursively.
- **Node insertion and deletion**: insertion and deletion operations are implemented by extending the BST algorithms and performing the required rebalancing after each update, checking the balance_factor calculated. The operations which can occour can be rotations in left or right direction.
- **Tree rotations**: rotations are used as balancing operations to restore the AVL property after insertions and deletions. These operations are partially inherited from the BST class. Single and double rotations are selected according to the imbalance configuration detected by the balance factor.

RBT (Red-Black Trees)

- **RBTNode implementation**: *RBTNode* class has been created to extend *Node* class contained in BST. Each node in fact stores an additional *color* field, which can be either red or black, fundamental to maintain the red-black tree properties.
- **Color management**: utility functions have been implemented to get, set and invert node colors. Missing children are considered black, simplifying the handling of boundary cases during insertion and deletion.
- **Node insertion and deletion**: insertion and deletion are implemented by extending the standard BST functions. After each insertion or deletion, a rebalancing procedure is performed, involving recoloring operations and tree rotations. The required operations restore the red-black properties while preserving the BST ordering.
- **Tree rotations**: left and right rotations inherited from BST are reused during the balancing procedures to modify the tree structure while preserving the ordering property.

5 Test Cases Setup

5.1 Choosing appropriate input sizes

The first issue to manage during the implementation of the test cases was the choice of the input sizes on which the experiments would be performed. We chose a wide range of tree sizes to make the simulation as realistic as possible, highlighting the performance differences among the different data structures.

A valid range of values for the number of nodes had to allow the analysis in multiple orders of magnitude. Instead of using a linear distribution of input sizes, which could have caused issues in tests on large numer of elements, a geometric progression was adopted in order to obtain a more uniform distribution on a logarithmic scale.

This approach allowed us to perform a better analysis of the growth trends of the different tree implementations.

These were the parameters selected:

$$n_{min} = 1000 \qquad n_{max} = 1\,000\,000 \qquad samples = 100$$

The list of input sizes is generated through the *calc_lista_val_n* function, which computes the values of n using a geometric progression. The common ratio of the progression is defined as:

$$n_i = \lfloor n_{min} \cdot c^i \rfloor \qquad c = \left(\frac{n_{max}}{n_{min}}\right)^{(\frac{1}{samples-1})}$$

The geometric progression produces a total of 100 samples, gradually increasing from the minimum value of nodes up to the maximum value. The first and last values of the sequence are reported below:

$$1000,\ 1072,\ 1150,\ 1233,\ 1322,\ \ldots,\ 811\,131,\ 869\,749,\ 932\,603, 1\,000\,000$$

Then for each n_i, we needed to choose how many test cases we want to make.

5.2 Choosing number of test cases

At the start we tried a constant number of test cases for each n_i. But then we quickly realized that this didn't make much sense because if the constant is too small, then

for larger n_i we wouldn't cover a good amount of cases, but if the constant is too big, then for smaller n_i we would make a lot of redundant test cases wasting time and resources.

So we changed it to be this function:

$$f(n) = min + k\sqrt{n} \tag{1}$$

The reason why we choose $\sqrt{n}$ is that it does not grow so big like n that it will make the testing very inefficient (think when n is 1000000) but it is still big enough to cover a lot of cases (on n = 1000000, 1000 cases at least if $k \geq 1$). The min is needed so that we do at least min tests for small n, and we choose k if we want to do more or less tests.

In the tests we choose $min = 100$ and $k = 2$ as it was a good compromise.

So the number of tests for each n_i is $100 + 2\sqrt{n_i}$.ù

5.3 Efficient Key Selection

5.4 Types of Tests Performed

After analyzing the given task we identified the possibility of extending the required experiment with additional measurements related to the structural and behavioral metrics of the trees. To ensure a meaningful evaluation, we designed three distinct experimental analyses.

5.4.1 Insertion Time Measurements

We needed to analyse the execution time required to perform insertion operations on the different tree implementations. For each value of n we first generated a random tree containing n nodes. Then, we performed a sequence of random insertions random insertions followed by deletions, as specified in the "Efficient Key Selection" paragraph above. The time required for each insertion operation was measured using Python's *perf_counter* function.

The final value considered in the analysis was the median of the measured times, providing a stable estimation of the insertion costs, reducing the influence of large variations in the measurements.

5.4.2 Rotations Count Measurements

As a first additional analysis we decided to verify the number of rotations performed by self-balancing trees during update operations, since we expected it to be closely related to the execution times observed.

Since every rotation operation is performed through the *rotate_left* and *rotate_right* methods, we decided to slightly modify these functions. We introduced a *rotations_counter* variable, initialized to 0 in the BST constructor, which is inherited by the other tree implementations. Then, every time a rotation is performed, the counter is incremented by one, allowing us to keep track of the total number of rotations executed.

After each insertion, the number of rotations executed during the operation was stored. The value considered for the experiment was the median number of rotations among all performed operations.

5.4.3 Heights Reached Measurements

In the second extension of the project, we analysed the height of the trees generated, comparing the behavior of each implemented data structure. The goal was to evaluate how effectively each tree maintains a low height during various updates.

To perform this analysis we used the fact that every implemented node type inherits its main features from the BST *Node* class, allowing the same height calculation method to be applied to all three data structure implementations.

We then implemented a recursive function called *height_ric*, which computes the height of a tree starting from a given *Node*. After each insertion operation we obtained the current height of the tree, starting from the root, and stored its value. At the end of each test, for each n, the maximum height obtained was considered to be the representative value.

5.5 Command-line Interface

For quick switching between different test configurations, we implemented a command-line interface based on flags.

It allows the user to choose which benchmarks to execute (insertions, rotations, and/or height measurements), the tree implementations to test (BST, AVL, and/or RBT), and the maximum number of nodes to be considered. This approach avoids

modifying the source code for each experiment and makes it easy to reproduce or customize the tests through simple command-line arguments.

5.6 Parallel Execution

From our initial analysis of the number of nodes to handle, it became evident that the execution performance would gradually degrade. The total volume of operations was huge, leading to execution times that were prohibitively long and inefficient for the amount of tests we needed to run.

Therefore, it was necessary to adopt a strategy to accelerate the process. We opted to distribute the computational workload across different CPU cores using Python's `ProcessPoolExecutor`. This tool allowed us to assign the analysis of each tree structure to a different processing unit concurrently.

As a result, the overall performance improved significantly. This simple but effective optimization reduced waiting times, allowing us to execute numerous tests.

5.7 Data Visualization and Export

To clearly visualize the results of our experiments we generated graphs using Python's `matplotlib` library. In these plots, the horizontal axis represents the number of nodes in the tree, while the vertical axis represents the specific metric being evaluated, such as the median execution time, the number of performed rotations or the maximum height of the tree. Each coloured line in the graph corresponds to a different tree data structure, making it easy to visually compare their behavior as the input size increases.

Furthermore, to avoid losing any generated data or overwriting previous, the plots are saved as PNG images uniquely distinguishing each filename with a precise timestamp. Alongside the plots, the exact numerical values are also exported into CSV files. This double-saving ensured the preservation of both an immediate visual synthesis of the growth trends and the raw data necessary for any further numerical review.

6 Results and Discussion

6.1 Test Machine and OS

In our initial approach to benchmarking the insertion times of BST, AVL, and RBT, we ran our test suites on standard desktop operating systems like Windows and full-featured Linux distributions. Because our tests required precision at the microsecond scale, we quickly discovered that these environments introduced significant noise into our data. Standard operating systems manage hundreds of concurrent background processes, forcing the OS scheduler to frequently preempt our process. These relentless context switches, and suspension of our process, created unpredictable timing spikes that heavily obscured the true algorithmic performance of the trees.

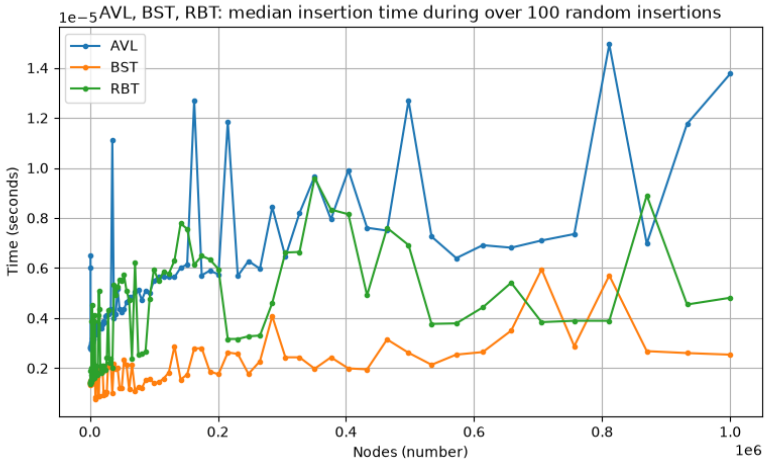

Figure 1: Example of test executed in Linux desktop

To eliminate this systemic jitter, we shifted our testing environment to a lightweight, command-line-only Linux distro (we choose Arch Linux). By stripping away the graphical user interface and unnecessary background daemons, we drastically reduced the operating system's footprint. We also executed all testing processes with maximum priority, ensuring our tests received the highest possible CPU time. This strictly controlled environment successfully suppressed external interference and scheduler interruptions, yielding highly stable and reproducible execution times. As a result, all final measurements used in this comparative analysis were exclusively gathered under this optimized Arch Linux setup.

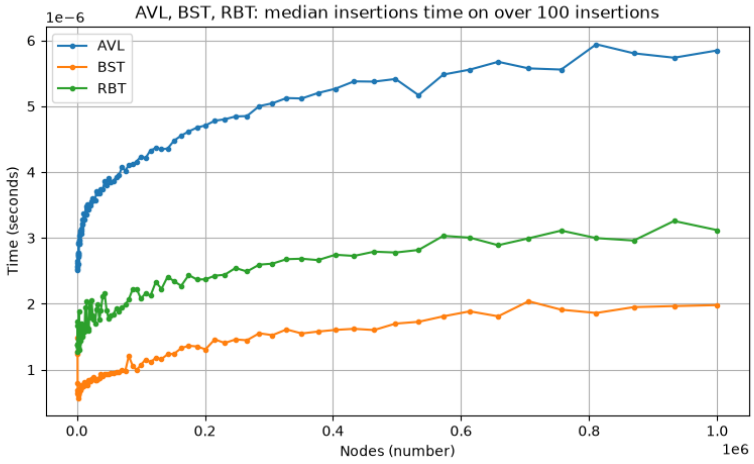

Figure 2: Example of test executed in Archlinux with maximum priority

Then for the hardware, we used a laptop always in charging with the following components:

- CPU: Intel Core i5-1335U
- RAM: 16GB DDR4-3200 MHz SO-DIMM Dual Channel
- Disk: Samsung SSD 980 500 GB

6.2 Insertions Tests

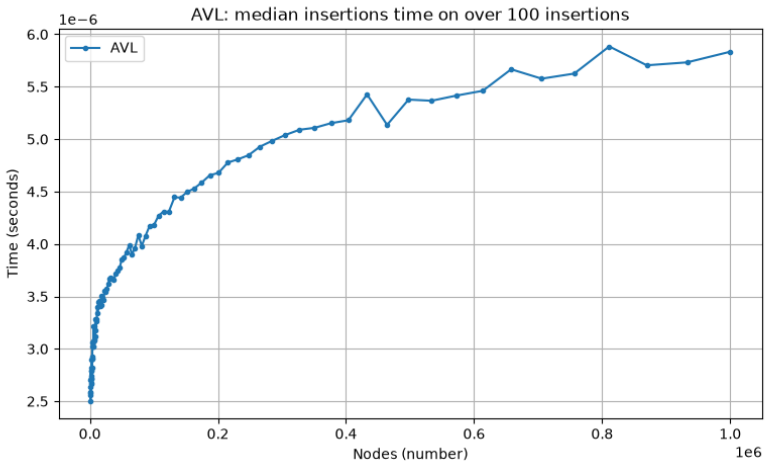

Figure 3: AVL insertion test

6.3 Rotations Count Tests

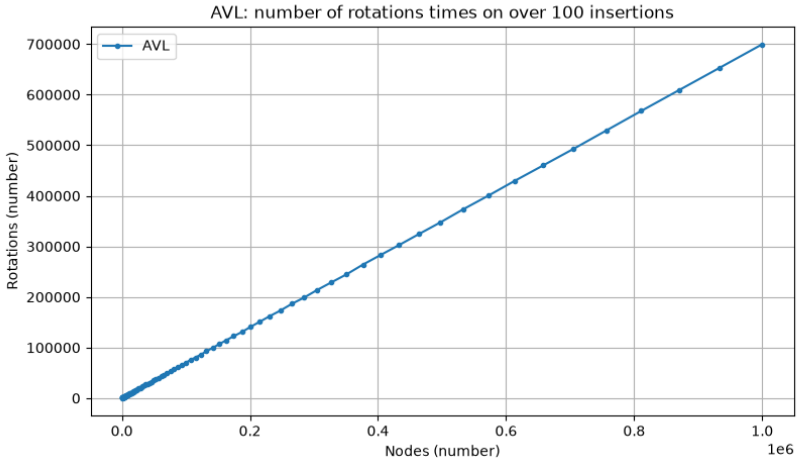

Figure 4: AVL rotation test

6.4 Heights Tests

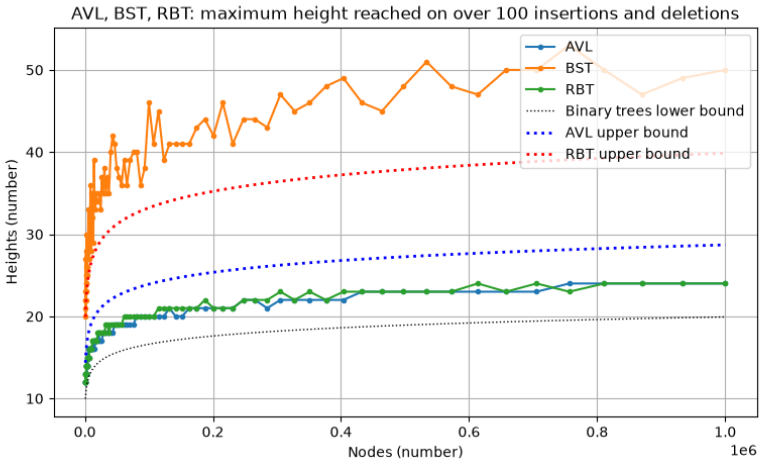

Figure 5: AVL rotation test

7 Conclusions

To conclude this project, we compared all the data structures we analyzed and drew conclusions regarding the ideal use cases for each type of tree.

7.1 Comparison of the Data Structures Analyzed

In this subsection, we analyze, for each tree, their strengths and weaknesses

- **BST**: we use this type of tree where fast insertion and deletion are the primary requirements and it is a necessity to avoid the computational overhead of rebalancing (even if the asymptotic time is the same, in real implementations the overhead does not amount to nothing). The data fed into the BST must be fed at "random", as to balance the BST naturally.
- **AVL**: we use this tree when we want to perform fast search lookups, as this is the tree that yields the lowest height, capping at $1.44 \cdot \log_2(n+2)$ (cite proof). The main drawback of the tree is insertion time, as the rotations provide a lot of overhead, but this overhead is necessary to achieve better search performance.
- **RBT**: this balanced data structure is an excellent compromise between the two trees discussed above. They are particularly efficient during insertion, thanks to a careful and strictly limited use of rotations during the balancing procedures. Although this means the tree might not achieve the absolute minimum height, the search performance remains highly competitive, with an upper bound of $2 \cdot \log_2(n+1)$ (cite proof). Since it combines a fast insertion akin to a BST and a search almost as optimal as an AVL, it is suitable in systems requiring frequent searches and updates.

7.2 Final Remarks

The comparison demonstrates that the additional complexity required to implement self-balancing trees is justified by the significant longterm gains in efficiency and robustness offered by these data structures. While a standard BST provides a simpler implementation and execution, it remains vulnerable to worst-case scenarios making it less reliable for large-scale datasets. Instead, choosing between an AVL and a RBT allows us to optimize the system based on whether the specific application requires fast searches or a balanced mix of frequent updates and lookups.

Bibliography & Sitography

[1] *CFS Scheduler - The Linux Kernel documentation*. Archived on `archive.org` on April 24th, 2026. URL: `https://web.archive.org/web/20260424153328/https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html#the-rbtree`.

[2] *std::set - cppreference.com*. Archived on `archive.org` on July 8th, 2026. URL: `https://web.archive.org/web/20260708070457/https://en.cppreference.com/cpp/container/set`.